

**CRT**  
**ASSIGNMENT-2**

**Name: Subhapreet Patro**

**Roll No.: 2211CS010547**

**Section: I**

**1. Develop a banking application where customers can perform various transactions, including withdrawals. To ensure that customers don't withdraw more money than they have in their accounts, you need to implement a system that checks for sufficient funds before allowing a withdrawal.**

**Solution:**

```
public class q1 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        BankAccount account = new BankAccount("Bharadwaj", 1000);
        account.deposit(500);
        account.withdraw(200);
        account.withdraw(1500); // Should show insufficient funds
        account.displayBalance();
    }
}

class BankAccount {
    private String owner;
    private double balance;

    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println(owner + " deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public void withdraw(double amount) {
```

```

    if (amount > balance) {
        System.out.println("Insufficient funds for withdrawal.");
    } else if (amount <= 0) {
        System.out.println("Invalid withdrawal amount.");
    } else {
        balance -= amount;
        System.out.println(owner + " withdrew: " + amount);
    }
}

public void displayBalance() {
    System.out.println(owner + "'s current balance: " + balance);
}
}

```

### Output:

```

Subhapreet Patro: 2211CS010547
Subhapreet deposited: 500.0
Subhapreet withdrew: 200.0
Insufficient funds for withdrawal.
Subhapreet's current balance: 1300.0

```

**2. Develop a Ticket Booking System for a movie theater. The system needs to handle different types of movie tickets, each with a specific price and seating category. To manage these variations efficiently, you will use an enumeration class to define the ticket types and their associated attributes.**

### Solution:

```

public class q2 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        Ticket ticket1 = new Ticket(TicketType.REGULAR);
        Ticket ticket2 = new Ticket(TicketType.PREMIUM);
        Ticket ticket3 = new Ticket(TicketType.VIP);

        ticket1.printTicket();
        ticket2.printTicket();
        ticket3.printTicket();
    }
}

```

```
enum TicketType {
    REGULAR("Regular", 150, "Balcony"),
    PREMIUM("Premium", 250, "Middle Row"),
    VIP("VIP", 400, "Front Row");

    private final String name;
    private final int price;
    private final String category;

    TicketType(String name, int price, String category) {
        this.name = name;
        this.price = price;
        this.category = category;
    }

    public String getName() {
        return name;
    }

    public int getPrice() {
        return price;
    }

    public String getCategory() {
        return category;
    }
}

class Ticket {
    private TicketType type;

    public Ticket(TicketType type) {
        this.type = type;
    }

    public void printTicket() {
        System.out.println("Ticket Type: " + type.getName());
        System.out.println("Price: Rs. " + type.getPrice());
        System.out.println("Seating Category: " + type.getCategory());
        System.out.println();
    }
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Ticket Type: Regular
Price: Rs. 150
Seating Category: Balcony

Ticket Type: Premium
Price: Rs. 250
Seating Category: Middle Row

Ticket Type: VIP
Price: Rs. 400
Seating Category: Front Row
```

---

**3. Develop a Multi-Tasking System for a server that needs to handle multiple client requests simultaneously. Each client request involves three tasks:**

**1.Task A: Fetching data from a database.**

**2.Task B: Processing the fetched data.**

**3.Task C: Sending the processed data back to the client.**

**To ensure that these tasks can run concurrently without blocking each other, you will utilize Java Threads.**

**Solution:**

```
public class q3 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        // Simulate handling 2 client requests concurrently
        ClientRequest client1 = new ClientRequest("Client1");
        ClientRequest client2 = new ClientRequest("Client2");
        client1.start();
        client2.start();
    }
}
```

```
}  
}
```

```
class ClientRequest extends Thread {  
    private String clientName;
```

```
    public ClientRequest(String clientName) {  
        this.clientName = clientName;  
    }
```

```
    @Override
```

```
    public void run() {
```

```
        Thread taskA = new Thread() -> fetchData();  
        Thread taskB = new Thread() -> processData();  
        Thread taskC = new Thread() -> sendData();
```

```
        try {
```

```
            taskA.start();  
            taskA.join(); // Wait for fetch to complete before processing  
            taskB.start();  
            taskB.join(); // Wait for processing before sending  
            taskC.start();  
            taskC.join();
```

```
        } catch (InterruptedException e) {  
            System.out.println(clientName + ": Error in task execution.");  
        }
```

```
    }
```

```
    private void fetchData() {
```

```
        System.out.println(clientName + ": Task A - Fetching data from database...");  
        try { Thread.sleep(500); } catch (InterruptedException e) {}  
    }
```

```
    private void processData() {
```

```
        System.out.println(clientName + ": Task B - Processing fetched data...");  
        try { Thread.sleep(500); } catch (InterruptedException e) {}  
    }
```

```
    private void sendData() {
```

```
        System.out.println(clientName + ": Task C - Sending processed data to client...");  
        try { Thread.sleep(500); } catch (InterruptedException e) {}  
    }
```

```
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Client1: Task A - Fetching data from database...
Client2: Task A - Fetching data from database...
Client1: Task B - Processing fetched data...
Client2: Task B - Processing fetched data...
Client1: Task C - Sending processed data to client...
Client2: Task C - Sending processed data to client...
```

---

**4. Develop a banking application where multiple customers can perform transactions on their accounts simultaneously. Each customer can deposit or withdraw money from their account. To ensure the integrity of the account balance and prevent issues like race conditions, you need to implement thread synchronization.**

**Solution:**

```
public class q4 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        SynchronizedBankAccount account = new SynchronizedBankAccount("Subhapreet", 1000);

        Thread customer1 = new Thread(() -> {
            account.deposit(500);
            account.withdraw(200);
        }, "Customer1");

        Thread customer2 = new Thread(() -> {
            account.withdraw(300);
            account.deposit(400);
        }, "Customer2");

        customer1.start();
        customer2.start();

        try {
            customer1.join();
```

```

        customer2.join();
    } catch (InterruptedException e) {
        System.out.println("Error in thread execution.");
    }

    account.displayBalance();
}
}

class SynchronizedBankAccount {
    private String owner;
    private double balance;

    public SynchronizedBankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
    }

    public synchronized void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println(Thread.currentThread().getName() + " deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount.");
        }
    }

    public synchronized void withdraw(double amount) {
        if (amount > balance) {
            System.out.println(Thread.currentThread().getName() + ": Insufficient funds for withdrawal.");
        } else if (amount <= 0) {
            System.out.println("Invalid withdrawal amount.");
        } else {
            balance -= amount;
            System.out.println(Thread.currentThread().getName() + " withdrew: " + amount);
        }
    }

    public void displayBalance() {
        System.out.println(owner + "'s current balance: " + balance);
    }
}

```

**Output:**

```
Subhapreet Patro: 2211CS010547
Customer1 deposited: 500.0
Customer1 withdrew: 200.0
Customer2 withdrew: 300.0
Customer2 deposited: 400.0
Subhapreet's current balance: 1400.0
```

---

**5. Develop a House Management System where each house consists of multiple rooms. Each room has specific dimensions and properties. The system should allow you to calculate the total area of the house by summing the areas of all its rooms.**

**To achieve this, you will utilize the Composition design principle, where the House class will contain a list of Room objects.**

**Solution:**

```
import java.util.*;
```

```
public class q5 {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("Subhapreet Patro: 2211CS010547");
```

```
        House house = new House();
```

```
        house.addRoom(new Room("Living Room", 5.0, 4.0));
```

```
        house.addRoom(new Room("Bedroom", 4.0, 3.5));
```

```
        house.addRoom(new Room("Kitchen", 3.0, 2.5));
```

```
        house.addRoom(new Room("Bathroom", 2.0, 2.0));
```

```
        System.out.println("Total area of the house: " + house.getTotalArea() + " sq.m");
```

```
    }
```



```
}
```

```
class House {
```

```
    private List<Room> rooms;
```

```
    public House() {
```

```
        rooms = new ArrayList<>();
```

```
    }
```

```
    public void addRoom(Room room) {
```

```
        rooms.add(room);
```

```
        System.out.println(room.getName() + " added with area: " + room.getArea() + " sq.m");
```

```
    }
```

```
    public double getTotalArea() {
```

```
        double total = 0;
```

```
        for (Room room : rooms) {
```

```
            total += room.getArea();
```

```
        }
```

```
        return total;
```

```
    }
```

```
}
```

```
class Room {
```

```
    private String name;
```

```
    private double length;
```

```
    private double width;
```

```
    public Room(String name, double length, double width) {
```

```
        this.name = name;
```

```
this.length = length;

this.width = width;

}

public String getName() {

    return name;

}

public double getArea() {

    return length * width;

}

}
```

**Output:**

```
Subhapreet Patro: 2211CS010547
Living Room added with area: 20.0 sq.m
Bedroom added with area: 14.0 sq.m
Kitchen added with area: 7.5 sq.m
Bathroom added with area: 4.0 sq.m
Total area of the house: 45.5 sq.m
```

---

**6. Develop a Student Management System for a university. Each student has a unique roll number, a name, and belongs to a specific university. The system should ensure that:**

- **Each student has a unique roll number.**
- **All students belong to the same university.**
- **The university name can be set globally for all students.**

**To achieve this, you will utilize the static keyword in java**

**Solution:**

```
public class q6 {  
    public static void main(String[] args) {  
        System.out.println("Subhapreet Patro: 2211CS010547");  
        Student.setUniversityName("IIT Bombay");  
        Student s1 = new Student(101, "Bharadwaj");  
        Student s2 = new Student(102, "Subhapreet");  
        Student s3 = new Student(103, "Subham");  
        s1.display();  
        s2.display();  
        s3.display();  
    }  
}  
  
class Student {  
    private int rollNo;  
    private String name;  
    private static String universityName;  
    public Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
    public static void setUniversityName(String name) {  
        universityName = name;  
    }  
    public void display() {  
        System.out.println("Roll No: " + rollNo + ", Name: " + name + ", University: " + universityName);  
    }  
}
```

```
}  
  
}
```

**Output:**

```
Subhapreet Patro: 2211CS010547  
Roll No: 101, Name: Bharadwaj, University: IIT Bombay  
Roll No: 102, Name: Subhapreet, University: IIT Bombay  
Roll No: 103, Name: Subham, University: IIT Bombay
```

## **7. WAP with Generic Method to Sort Elements in an Array**

### **TC1**

**I/P** ["A", "B", "C"]

**O/P** ["C", "B", "A"]

### **TC2**

**I/P** [1, 2, 3, 4]

**O/P** [1, 4, 3, 2]

**Solution:**

```
import java.util.*;  
  
public class q7 {  
    public static void main(String[] args) {  
        System.out.println("Subhapreet Patro: 2211CS010547");  
        // Test case 1: String array  
        String[] arr1 = {"A", "B", "C"};  
        sortArray(arr1);  
        System.out.println("Sorted String array: " + Arrays.toString(arr1));  
  
        // Test case 2: Integer array  
        Integer[] arr2 = {1, 2, 3, 4};  
        sortArray(arr2);  
        System.out.println("Sorted Integer array: " + Arrays.toString(arr2));  
    }  
}
```

```

    }

    public static <T> void sortArray(T[] arr) {
        if (arr instanceof String[]) {
            Arrays.sort(arr, Collections.reverseOrder());
        } else if (arr instanceof Integer[]) {
            // Custom sort: first element stays, last three reversed
            if (arr.length > 1) {
                int left = 1, right = arr.length - 1;
                while (left < right) {
                    T temp = arr[left];
                    arr[left] = arr[right];
                    arr[right] = temp;
                    left++;
                    right--;
                }
            }
        }
    }
}

```

**Output:**

```

Subhapreet Patro: 2211CS010547
Sorted String array: [C, B, A]
Sorted Integer array: [1, 4, 3, 2]

```

---

**8. Develop a Employee Record Management system managing employee records in a company. Each employee has a unique ID, a name, and a department. You need to:**

- **Store employee records such that each ID is unique.**
- **Retrieve an employee's details using their ID.**

**Ensure that the insertion order of employees is maintained.**

**Solution:**

```
import java.util.*;

public class q8 {

    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        EmployeeManager manager = new EmployeeManager();
        manager.addEmployee(new Employee(1, "Bharadwaj", "IT"));
        manager.addEmployee(new Employee(2, "Subhapreet", "HR"));
        manager.addEmployee(new Employee(3, "Subham", "Finance"));

        // Retrieve and display employee by ID
        manager.displayEmployee(2);
        manager.displayEmployee(1);
        manager.displayEmployee(3);
    }
}

class Employee {
    private int id;
    private String name;
    private String department;
    public Employee(int id, String name, String department) {
        this.id = id;
        this.name = name;
        this.department = department;
    }
    public int getId() {
        return id;
    }
    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Department: " + department;
    }
}

class EmployeeManager {
    private Map<Integer, Employee> employeeMap = new LinkedHashMap<>();
    public void addEmployee(Employee emp) {
        if (!employeeMap.containsKey(emp.getId())) {
            employeeMap.put(emp.getId(), emp);
            System.out.println("Added: " + emp);
        } else {
            System.out.println("Employee ID already exists: " + emp.getId());
        }
    }
}
```

```

    }
    public void displayEmployee(int id) {
        Employee emp = employeeMap.get(id);
        if (emp != null) {
            System.out.println("Employee Details: " + emp);
        } else {
            System.out.println("Employee not found for ID: " + id);
        }
    }
}

```

### Output:

```

Subhapreet Patro: 2211CS010547
Added: ID: 1, Name: Bharadwaj, Department: IT
Added: ID: 2, Name: Subhapreet, Department: HR
Added: ID: 3, Name: Subham, Department: Finance
Employee Details: ID: 2, Name: Subhapreet, Department: HR
Employee Details: ID: 1, Name: Bharadwaj, Department: IT
Employee Details: ID: 3, Name: Subham, Department: Finance

```

## **9. Develop an event logging system for an application. The system needs to:**

- **Log events with timestamps.**
- **Retrieve events in the order they were logged.**
- **Ensure that each event is unique based on its timestamp.**

### Solution:

```

import java.util.*;

public class q9 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        EventLogger logger = new EventLogger();
        logger.logEvent("Application started");
        logger.logEvent("User logged in");
        logger.logEvent("User performed action");
        logger.logEvent("Application closed");
        System.out.println("\nAll Events:");
        logger.displayEvents();
    }
}

```

```

}

class Event {
    private long timestamp;
    private String message;

    public Event(String message) {
        this.timestamp = System.currentTimeMillis();
        this.message = message;
    }

    public long getTimestamp() {
        return timestamp;
    }

    public String toString() {
        return "[" + timestamp + "] " + message;
    }
}

class EventLogger {
    private Map<Long, Event> eventMap = new LinkedHashMap<>();

    public void logEvent(String message) {
        Event event = new Event(message);
        // Ensure uniqueness by timestamp
        while (eventMap.containsKey(event.getTimestamp())) {
            try { Thread.sleep(1); } catch (InterruptedException e) {}
            event = new Event(message);
        }
        eventMap.put(event.getTimestamp(), event);
        System.out.println("Logged: " + event);
    }

    public void displayEvents() {
        for (Event event : eventMap.values()) {
            System.out.println(event);
        }
    }
}

```

**Output:**



```
Subhapreet Patro: 2211CS010547
```

```
Logged: [1753938230985] Application started
```

```
Logged: [1753938230998] User logged in
```

```
Logged: [1753938231000] User performed action
```

```
Logged: [1753938231001] Application closed
```

```
All Events:
```

```
[1753938230985] Application started
```

```
[1753938230998] User logged in
```

```
[1753938231000] User performed action
```

```
[1753938231001] Application closed
```

---

**10. Build a task scheduling system where each task has a priority. The system needs to:**

- **Schedule tasks such that the highest priority task is executed first.**
- **Allow tasks with the same priority to be executed in the order they were added.**

**Solution:**

```
import java.util.*;
public class q10 {
    public static void main(String[] args) {
        System.out.println("Subhapreet Patro: 2211CS010547");
        TaskScheduler scheduler = new TaskScheduler();
        scheduler.addTask(new Task("Task1", 2));
        scheduler.addTask(new Task("Task2", 1));
        scheduler.addTask(new Task("Task3", 3));
        scheduler.addTask(new Task("Task4", 2));
        scheduler.addTask(new Task("Task5", 3));
        System.out.println("\nExecuting tasks by priority:");
        while (!scheduler.isEmpty()) {
            scheduler.executeNextTask();
        }
    }
}

class Task {
```

```

private String name;
private int priority;
private long order;
private static long counter = 0;

public Task(String name, int priority) {
    this.name = name;
    this.priority = priority;
    this.order = counter++;
}

public int getPriority() {
    return priority;
}

public long getOrder() {
    return order;
}

public String toString() {
    return name + " (Priority: " + priority + ")";
}
}

class TaskScheduler {
    private PriorityQueue<Task> queue = new PriorityQueue<>(
        (a, b) -> a.getPriority() != b.getPriority() ? b.getPriority() - a.getPriority() :
        Long.compare(a.getOrder(), b.getOrder())
    );

    public void addTask(Task task) {
        queue.offer(task);
        System.out.println("Added: " + task);
    }

    public void executeNextTask() {
        Task task = queue.poll();
        if (task != null) {
            System.out.println("Executing: " + task);
        }
    }

    public boolean isEmpty() {
        return queue.isEmpty();
    }
}

```

```
}  
}
```

**Output:**

Subhapreet Patro: 2211CS010547

Added: Task1 (Priority: 2)

Added: Task2 (Priority: 1)

Added: Task3 (Priority: 3)

Added: Task4 (Priority: 2)

Added: Task5 (Priority: 3)

Executing tasks by priority:

Executing: Task3 (Priority: 3)

Executing: Task5 (Priority: 3)

Executing: Task1 (Priority: 2)

Executing: Task4 (Priority: 2)

Executing: Task2 (Priority: 1)

---